

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
%matplotlib inline
mpl.style.use('ggplot')
```

```
car=pd.read_csv('quikr_car.csv')
```

```
car.head()
```

	name	company	year	Price	kms_driven	fuel_type
0	Hyundai Santro Xing XO eRLX Euro III	Hyundai	2007	80,000	45,000 kms	Petrol
1	Mahindra Jeep CL550 MDI	Mahindra	2006	4,25,000	40 kms	Diesel
2	Maruti Suzuki Alto 800 Vxi	Maruti	2018	Ask For Price	22,000 kms	Petrol
3	Hyundai Grand i10 Magna 1.2 Kappa VTVT	Hyundai	2014	3,25,000	28,000 kms	Petrol
4	Ford EcoSport Titanium 1.5L TDCi	Ford	2014	5,75,000	36,000 kms	Diesel

```
car.shape
```

```
(892, 6)
```

```
car.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 892 entries, 0 to 891
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        892 non-null    object
1   company     892 non-null    object
2   year        892 non-null    object
3   Price       892 non-null    object
4   kms_driven  840 non-null    object
5   fuel_type   837 non-null    object
dtypes: object(6)
memory usage: 41.9+ KB
```

```
backup=car.copy()
```

```
car=car[car['year'].str.isnumeric()]
```

```
car['year']=car['year'].astype(int)
```

```
car=car[car['Price']!='Ask For Price']
```

```
car['Price']=car['Price'].str.replace(',','').astype(int)
```

```
car['kms_driven']=car['kms_driven'].str.split().str.get(0).str.replace(',','')
```

```
car=car[car['kms_driven'].str.isnumeric()]
```

```
car['kms_driven']=car['kms_driven'].astype(int)
```

```
car=car[~car['fuel_type'].isna()]
```

```
car.shape
```

```
(816, 6)
```

```
car['name']=car['name'].str.split().str.slice(start=0,stop=3).str.join(' ')
```

```
car=car.reset_index(drop=True)
```

car

	name	company	year	Price	kms_driven	fuel_type
0	Hyundai Santro Xing	Hyundai	2007	80000	45000	Petrol
1	Mahindra Jeep CL550	Mahindra	2006	425000	40	Diesel
2	Hyundai Grand i10	Hyundai	2014	325000	28000	Petrol
3	Ford EcoSport Titanium	Ford	2014	575000	36000	Diesel
4	Ford Figo	Ford	2012	175000	41000	Diesel
...	...	...	...	...	...	...
811	Maruti Suzuki Ritz	Maruti	2011	270000	50000	Petrol
812	Tata Indica V2	Tata	2009	110000	30000	Diesel
813	Toyota Corolla Altis	Toyota	2009	300000	132000	Petrol
814	Tata Zest XM	Tata	2018	260000	27000	Diesel
815	Mahindra Quanto C8	Mahindra	2013	390000	40000	Diesel

816 rows × 6 columns

```
car.to_csv('Cleaned_Car_data.csv')
```

```
car.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 816 entries, 0 to 815
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0    name        816 non-null    object
1    company     816 non-null    object
2    year        816 non-null    int64
3    Price       816 non-null    int64
4    kms_driven  816 non-null    int64
5    fuel_type   816 non-null    object
dtypes: int64(3), object(3)
memory usage: 38.4+ KB
```

```
car.describe(include='all')
```

	name	company	year	Price	kms_driven	fuel_type
count	816	816	816.000000	8.160000e+02	816.000000	816
unique	254	25	NaN	NaN	NaN	3
top	Maruti Suzuki Swift	Maruti	NaN	NaN	NaN	Petrol
freq	51	221	NaN	NaN	NaN	428
mean	NaN	NaN	2012.444853	4.117176e+05	46275.531863	NaN
std	NaN	NaN	4.002992	4.751844e+05	34297.428044	NaN
min	NaN	NaN	1995.000000	3.000000e+04	0.000000	NaN
25%	NaN	NaN	2010.000000	1.750000e+05	27000.000000	NaN
50%	NaN	NaN	2013.000000	2.999990e+05	41000.000000	NaN
75%	NaN	NaN	2015.000000	4.912500e+05	56818.500000	NaN
max	NaN	NaN	2019.000000	8.500003e+06	400000.000000	NaN

```
car=car[car['Price']<6000000]
```

```
car['company'].unique()
```

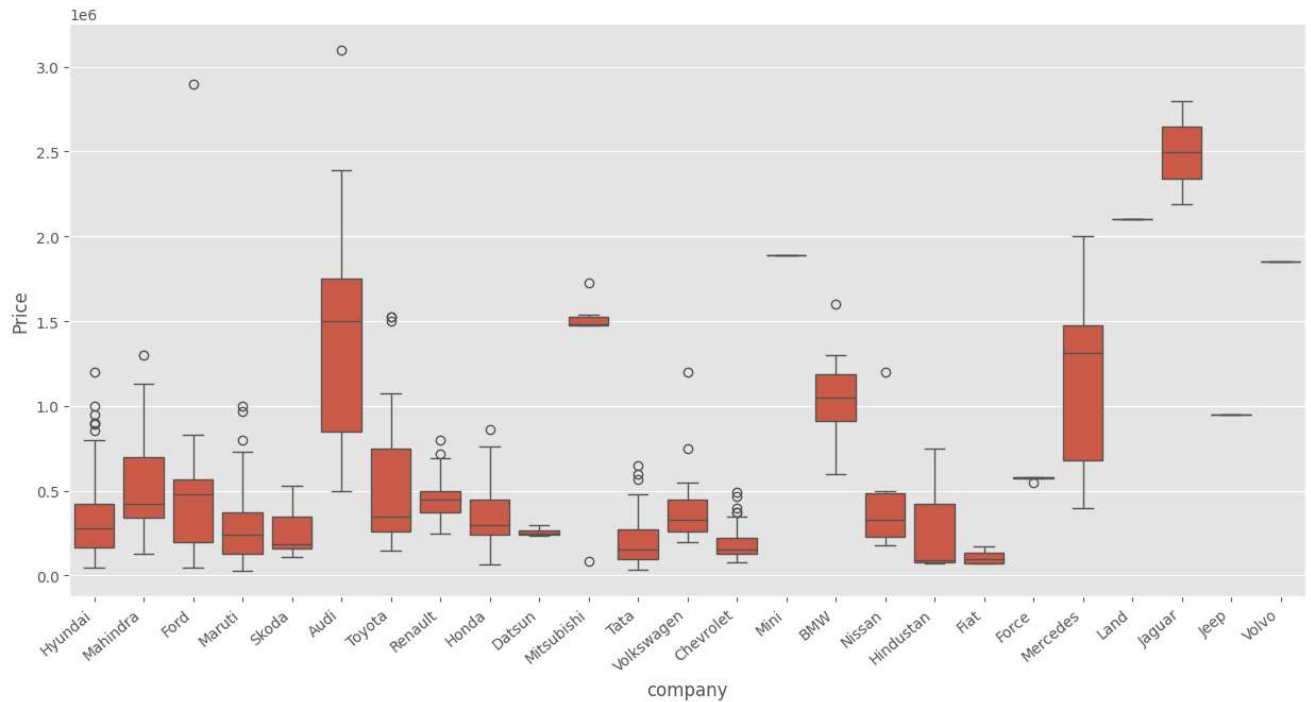
```
array(['Hyundai', 'Mahindra', 'Ford', 'Maruti', 'Skoda', 'Audi', 'Toyota',
      'Renault', 'Honda', 'Datsun', 'Mitsubishi', 'Tata', 'Volkswagen',
```

```
'Chevrolet', 'Mini', 'BMW', 'Nissan', 'Hindustan', 'Fiat', 'Force',
'Mercedes', 'Land', 'Jaguar', 'Jeep', 'Volvo'], dtype=object)
```

```
import seaborn as sns
```

```
plt.subplots(figsize=(15,7))
ax=sns.boxplot(x='company',y='Price',data=car)
ax.set_xticklabels(ax.get_xticklabels(),rotation=40,ha='right')
plt.show()
```

/tmp/ipykernel_1272/2788130517.py:3: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after ax.set_xticklabels(ax.get_xticklabels(),rotation=40,ha='right')

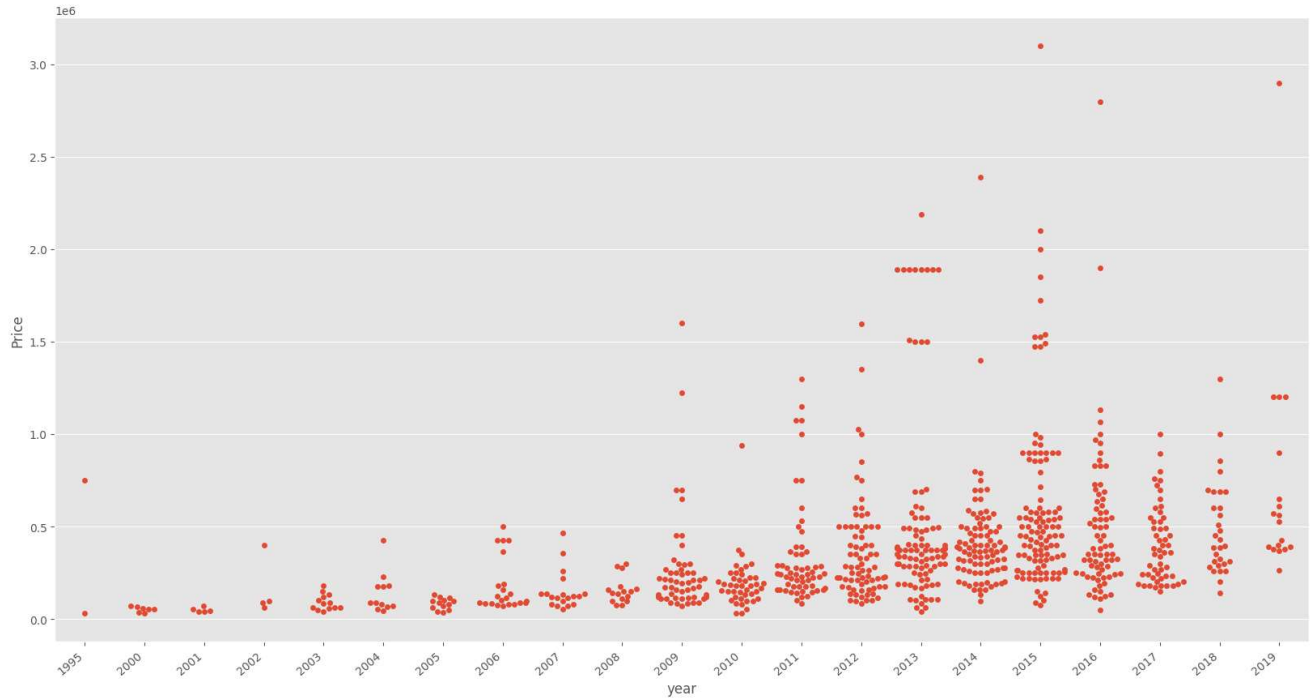


```
plt.subplots(figsize=(20,10))
ax=sns.swarmplot(x='year',y='Price',data=car)
ax.set_xticklabels(ax.get_xticklabels(),rotation=40,ha='right')
plt.show()
```

```

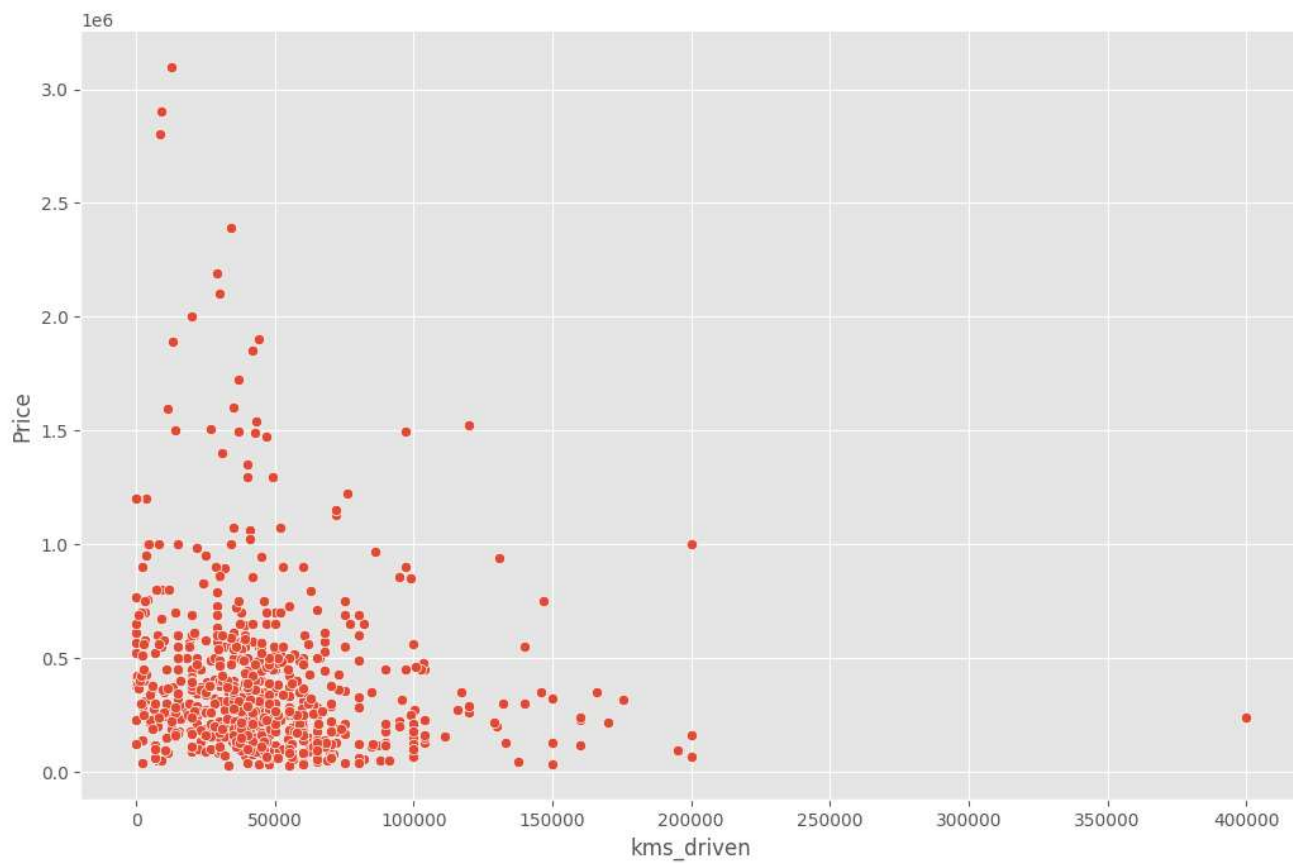
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 13.6% of the points cannot be placed; you may
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 13.0% of the points cannot be placed; you may
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 6.8% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 10.6% of the points cannot be placed; you may
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 7.7% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)
/tmp/ipykernel_1272/2542042277.py:3: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after
ax.set_xticklabels(ax.get_xticklabels(),rotation=40,ha='right')
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 9.3% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 6.8% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 9.6% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)
/usr/local/lib/python3.13/dist-packages/seaborn/categorical.py:3399: UserWarning: 5.5% of the points cannot be placed; you may w
warnings.warn(msg, UserWarning)

```



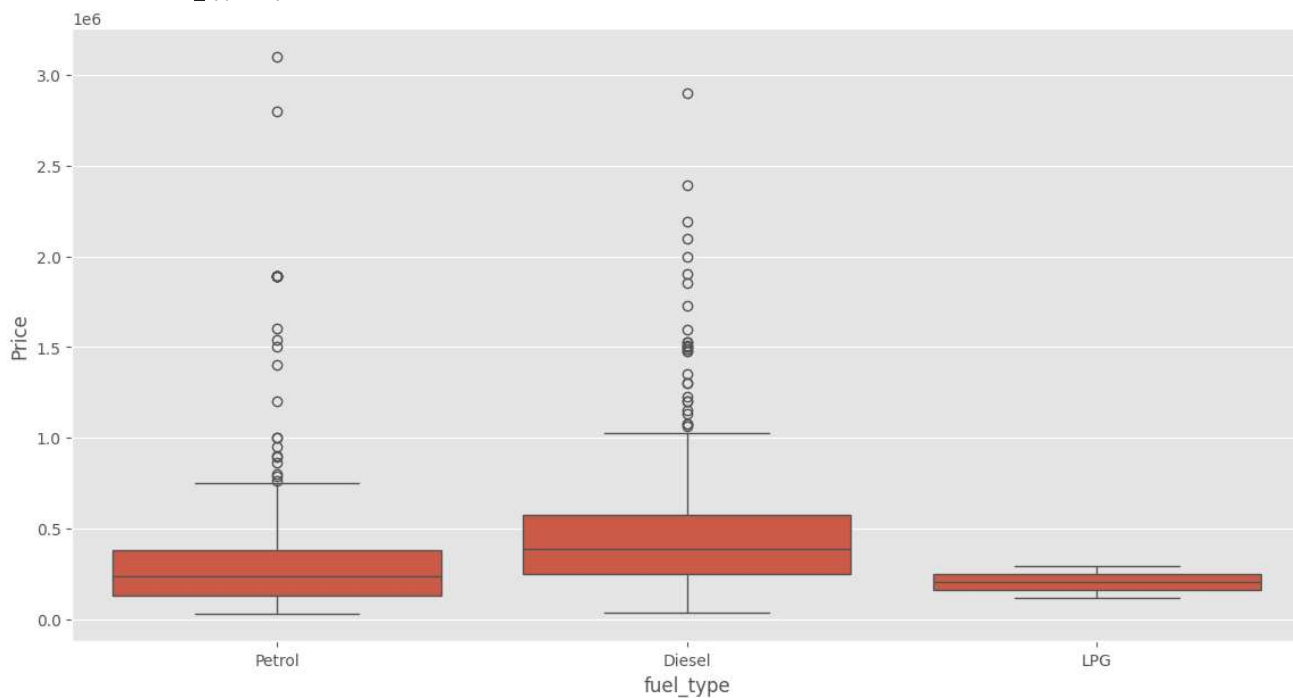
```
sns.relplot(x='kms_driven',y='Price',data=car,height=7,aspect=1.5)
```

```
<seaborn.axisgrid.FacetGrid at 0x7f7827163610>
```



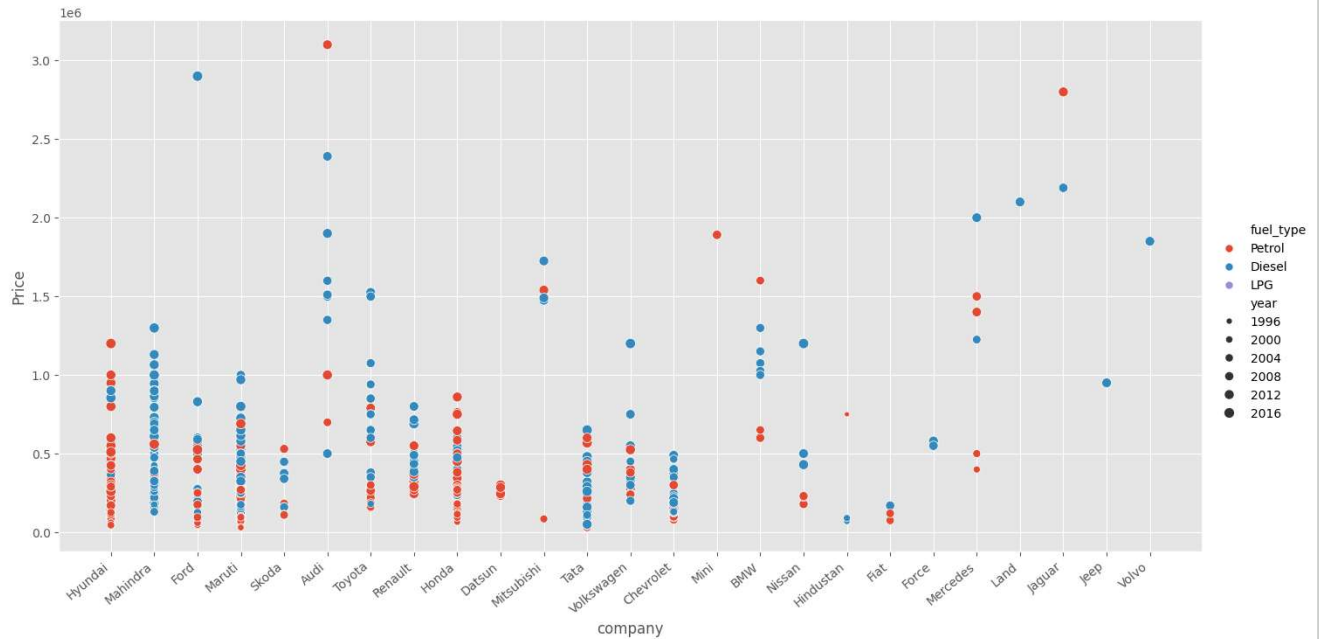
```
plt.subplots(figsize=(14,7))  
sns.boxplot(x='fuel_type',y='Price',data=car)
```

```
<Axes: xlabel='fuel_type', ylabel='Price'>
```



```
ax=sns.relplot(x='company',y='Price',data=car,hue='fuel_type',size='year',height=7,aspect=2)
ax.set_xticklabels(rotation=40,ha='right')
```

```
<seaborn.axisgrid.FacetGrid at 0x7f7827041450>
```



```
X=car[['name','company','year','kms_driven','fuel_type']]
y=car['Price']
```

X

	name	company	year	kms_driven	fuel_type
0	Hyundai Santro Xing	Hyundai	2007	45000	Petrol
1	Mahindra Jeep CL550	Mahindra	2006	40	Diesel
2	Hyundai Grand i10	Hyundai	2014	28000	Petrol
3	Ford EcoSport Titanium	Ford	2014	36000	Diesel
4	Ford Figo	Ford	2012	41000	Diesel
...	...	...	...	...	...
811	Maruti Suzuki Ritz	Maruti	2011	50000	Petrol
812	Tata Indica V2	Tata	2009	30000	Diesel
813	Toyota Corolla Altis	Toyota	2009	132000	Petrol
814	Tata Zest XM	Tata	2018	27000	Diesel
815	Mahindra Quanto C8	Mahindra	2013	40000	Diesel

815 rows × 5 columns

y.shape

(815,)

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score
```

```
ohe=OneHotEncoder()
ohe.fit(X[['name', 'company', 'fuel_type']])
```

▼ OneHotEncoder ⓘ ?

OneHotEncoder()

```
column_trans=make_column_transformer((OneHotEncoder(categories=ohe.categories_), ['name', 'company', 'fuel_type']),
                                     remainder='passthrough')
```

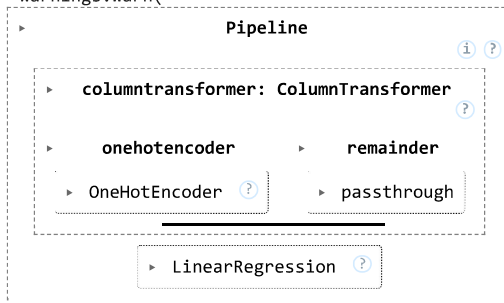
```
lr=LinearRegression()
```

```
pipe=make_pipeline(column_trans,lr)
```

```
pipe.fit(X_train,y_train)
```

/usr/local/lib/python3.13/dist-packages/sklearn/compose/_column_transformer.py:1667: FutureWarning:
The format of the columns of the 'remainder' transformer in ColumnTransformer.transformers_ will change in version 1.7 to match
At the moment the remainder columns are stored as indices (of type int). With the same ColumnTransformer configuration, in the f
To use the new behavior now and suppress this warning, use ColumnTransformer(force_int_remainder_cols=False).

```
warnings.warn(
```



```
y_pred=pipe.predict(X_test)
```

```
r2_score(y_test,y_pred)
```

```
0.507553475466459
```

```
scores=[]
for i in range(1000):
    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.1,random_state=i)
    lr=LinearRegression()
    pipe=make_pipeline(column_trans,lr)
    pipe.fit(X_train,y_train)
    y_pred=pipe.predict(X_test)
    scores.append(r2_score(y_test,y_pred))
```

```
np.argmax(scores)
```

```
np.int64(302)
```

```
scores[np.argmax(scores)]
```

```
0.8991157554877304
```

```
pipe.predict(pd.DataFrame(columns=X_test.columns,data=np.array(['Maruti Suzuki Swift', 'Maruti', 2019, 100, 'Petrol']).reshape(1,5))
array([430301.37134528])
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.1,random_state=np.argmax(scores))
lr=LinearRegression()
```

```
pipe=make_pipeline(column_trans,lr)
pipe.fit(X_train,y_train)
y_pred=pipe.predict(X_test)
# score(y_test, y_pred)
```

```
0.8991157554877304
```

```
import pickle
```

```
pickle.dump(pipe,open('LinearRegressionModel.pkl','wb'))
```

```
pipe.predict(pd.DataFrame(columns=['name','company','year','kms_driven','fuel_type'],data=np.array(['Maruti Suzuki Swift','Maru
array([456670.3272301])
```

```
pipe.steps[0][1].transformers[0][1].categories[0]
```

```
array(['Audi A3 Cabriolet', 'Audi A4 1.8', 'Audi A4 2.0', 'Audi A6 2.0',
'Audi A8', 'Audi Q3 2.0', 'Audi Q5 2.0', 'Audi Q7', 'BMW 3 Series',
'BMW 5 Series', 'BMW 7 Series', 'BMW X1', 'BMW X1 sDrive20d',
'BMW X1 xDrive20d', 'Chevrolet Beat', 'Chevrolet Beat Diesel',
'Chevrolet Beat LS', 'Chevrolet Beat LT', 'Chevrolet Beat PS',
'Chevrolet Cruze LTZ', 'Chevrolet Enjoy', 'Chevrolet Enjoy 1.4',
'Chevrolet Sail 1.2', 'Chevrolet Sail UVA', 'Chevrolet Spark',
'Chevrolet Spark 1.0', 'Chevrolet Spark LS', 'Chevrolet Spark LT',
'Chevrolet Tavera LS', 'Chevrolet Tavera Neo', 'Datsun GO T',
'Datsun Go Plus', 'Datsun Redi GO', 'Fiat Linea Emotion',
'Fiat Petra ELX', 'Fiat Punto Emotion', 'Force Motors Force',
'Force Motors One', 'Ford EcoSport', 'Ford EcoSport Ambiente',
'Ford EcoSport Titanium', 'Ford EcoSport Trend',
'Ford Endeavor 4x4', 'Ford Fiesta', 'Ford Fiesta SXi', 'Ford Figo',
'Ford Figo Diesel', 'Ford Figo Duratorq', 'Ford Figo Petrol',
'Ford Fusion 1.4', 'Ford Ikon 1.3', 'Ford Ikon 1.6',
'Hindustan Motors Ambassador', 'Honda Accord', 'Honda Amaze',
'Honda Amaze 1.2', 'Honda Amaze 1.5', 'Honda Brio', 'Honda Brio V',
```